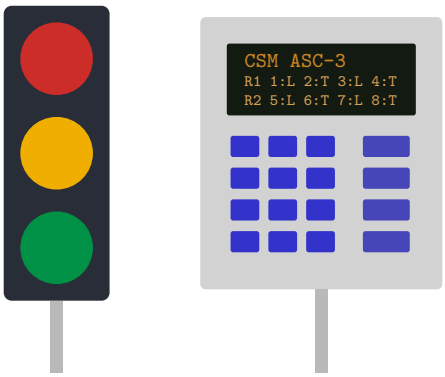
CSM ASC-3

Advanced Signal Controller

Operations Manual & Signal Timing Primer



Fundamentals of dual-ring actuated control
and a complete programming guide for the in-game ASC-3 panel

Minecraft City Super Mod (csm) ▪ July 2026

Contents

1 Introduction **3**
1.1 Where ADVANCED fits among the controller modes . . . 3
1.2 How to read this manual . . . 3

2 Fundamentals of Signal Control **3**
2.1 Movements and phases . . . 3
2.2 Rings and barriers . . . 4
2.3 Actuated timing: the life of a green . . . 5
2.4 Recalls and resting . . . 5
2.5 Pedestrian timing . . . 5

3 Getting Started **6**
3.1 What you need . . . 6
3.2 Enter ADVANCED mode and open the panel . . . 6
3.3 The 60-second program . . . 6
3.4 What “Load Std 8-Phase” actually programs . . . 6

4 The Programmer Panel **7**
4.1 Status LEDs . . . 7
4.2 Editing model . . . 7

5 Screen Reference **8**
5.1 STATUS . . . 8
5.2 TIMING — per-phase intervals . . . 8
5.3 MAP — what each phase drives . . . 8
5.4 ACT — actuation / volume-density . . . 10
5.5 COORD . . . 11
5.6 PREEMPT — preemption sequences . . . 11
5.7 OVL — vehicle overlaps . . . 11
5.8 HELP . . . 12

6 Coordination: Cycles, Splits, and Offsets **12**
6.1 Free vs. coordinated . . . 12
6.2 The cycle as a clock . . . 13
6.3 Splits are budgeted per barrier . . . 13
6.4 Dual entry under coordination . . . 14
6.5 Offsets and green waves . . . 15
6.6 Programming recipe . . . 16
6.7 Worked example . . . 16

7 Flashing Yellow Arrows (FYA) **16**
7.1 The hardware . . . 17
7.2 Programming . . . 17

8 Saving and Loading Whole Plans **17**
8.1 From the panel . . . 17
8.2 From the command line . . . 17
8.3 What a plan changes . . . 17

9 Cookbook **18**
9.1 Two one-way streets (the minimal program) . . . 18
9.2 Four-way crossing, no lefts . . . 18
9.3 Main street with protected + permissive lefts . . . 18
9.4 The full build: all 8 phases + right-turn FYAs . . . 18
9.5 Bicycles on the arterial . . . 18
9.6 An exclusive pedestrian phase (and ped overlaps) . . . 19
9.7 A railroad crossing . . . 19
9.8 A coordinated corridor . . . 19

10 Troubleshooting & FAQ **20**

A Glossary **20**

B Cross-reference to the codebase **21**

1 Introduction

The **CSM ASC-3** is the advanced operating mode of the mod's traffic signal controller block: a full *NEMA dual-ring, dual-barrier actuated controller*, styled after the real-world Econolite ASC/3. Where the controller's `NORMAL` mode services one circuit at a time, `ADVANCED` mode runs concurrent movements across two timing rings, with per-phase intervals, vehicle and pedestrian detection, recalls, coordination (green waves), preemption, flashing yellow arrows, and vehicle overlaps.

This manual is two things at once:

- a **primer** on how real traffic signal timing works — phases, rings, barriers, actuation, splits, offsets — because the ASC-3 models these concepts faithfully and they are the fastest way to understand it; and
- a **user guide** for the in-game programming panel: every screen, every field, and cookbook recipes for common intersections.

1.1 Where ADVANCED fits among the controller modes

The controller block supports several operating modes; the ASC-3 panel programs the last one. The others are documented in `assets/docs/TRAFFIC_SIGNAL_SYSTEM.md`.

Mode	Behavior
`NORMAL`	Classic circuit-at-a-time coordinated operation.
`FLASH`	Yellow/red flash; program which movement gets which on MAP (Section 5.3).
`REQUESTABLE`	Serves only on sensor/button request.
`RAMP_METER_*`	Freeway ramp metering (full/part time).
`WRONG_WAY_DETECTION`	TAPCO-style wrong-way beacon system.
`MANUAL_OFF`	All signals dark.
`ADVANCED`	**This manual:** NEMA dual-ring phase controller.

1.2 How to read this manual

If you are new to signal timing, read Section 2 first — everything on the panel maps to a concept there. If you just want a working intersection, jump to the quick start in Section 3 and the cookbook in Section 9. Section 5 is the field-by-field reference, and Section 6 is a deep dive on coordination.

2 Fundamentals of Signal Control

2.1 Movements and phases

A **movement** is one stream of traffic through the intersection: the eastbound through, the westbound left, a crosswalk. A **phase** is the controller's unit of right-of-way: a movement (or compatible set) plus all the timing that governs its green.

The ASC-3 uses the standard **NEMA 8-phase** numbering (FHWA convention). Even phases are the through movements; odd phases are the protected lefts. Each odd left is numbered so it sits in the ring *beside the opposing through* — $\varphi 1$ is the left that crosses $\varphi 2$ (it shares an approach with $\varphi 6$), $\varphi 5$ shares an approach with $\varphi 2$, and likewise $\varphi 3$ with $\varphi 8$ and $\varphi 7$ with $\varphi 4$. That numbering is what makes every legal cross-ring pair ($\varphi 1{+}\varphi 6$, $\varphi 2{+}\varphi 5$, …) a same-approach or dual-left combination instead of a conflict. A left's flashing-yellow-arrow reference is its *opposing* through: $\varphi 1 \rightarrow$ P2, $\varphi 5 \rightarrow$ P6, $\varphi 3 \rightarrow$ P4, $\varphi 7 \rightarrow$ P8 (see Section 7).

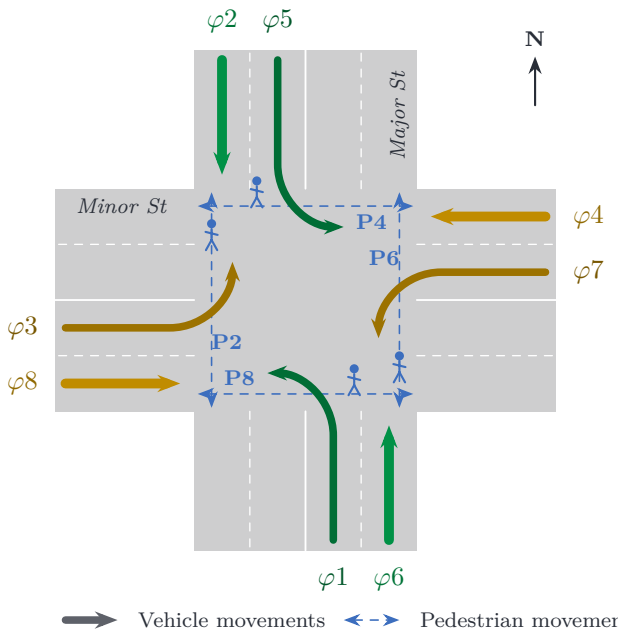


Figure 1: NEMA phase numbering as used by Load Std 8-Phase, per the FHWA convention. The main street runs north–south (green): southbound $\varphi 2$ with its left $\varphi 5$, northbound $\varphi 6$ with $\varphi 1$. The side street runs east–west (yellow): westbound $\varphi 4$ with $\varphi 7$, eastbound $\varphi 8$ with $\varphi 3$. Even = through, odd = protected left, and each left is numbered beside the *opposing* through. Pedestrian movements (dashed blue) carry the number of the through phase they walk with: P2/P6 cross the minor street alongside the main-street greens, P4/P8 cross the major street with the side-street greens.

2.2 Rings and barriers

The eight phases are arranged in two **rings** that time in parallel — at any moment, at most one phase per ring is green, so the intersection serves up to two compatible movements at once. A **barrier** divides the phases into the main-street group and the side-street group: both rings must finish their group and cross the barrier *together*, which is what keeps conflicting movements apart.

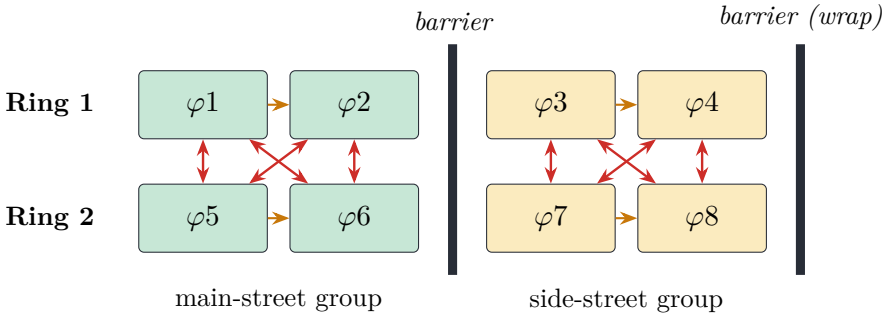


Figure 2: The dual-ring, dual-barrier structure. One green per ring, advancing left to right (amber arrows). The red double-headed arrows mark the compatible cross-ring pairs that may be green together: $\varphi 1{+}\varphi 5$, $\varphi 1{+}\varphi 6$, $\varphi 2{+}\varphi 5$, $\varphi 2{+}\varphi 6$ on the main street and $\varphi 3{+}\varphi 7$, $\varphi 3{+}\varphi 8$, $\varphi 4{+}\varphi 7$, $\varphi 4{+}\varphi 8$ on the side street — never a pair across a barrier. Rings cross a barrier only together.

Because each ring skips phases that have no demand, the structure is flexible: the dual lefts can lead together, one left can lag beside the opposing through, or the lefts can be skipped entirely — all without any explicit “pattern” programming.

2.3 Actuated timing: the life of a green

The ASC-3 is **actuated**: phases are served because something asked (a *call* from a vehicle sensor, a pedestrian button, or a programmed recall), and greens last only as long as the demand does. Each green obeys these intervals (all set per phase on the TIMING screen):

Min Green the guaranteed floor — the green never ends earlier.

Passage the extension timer ("gap"): each detected vehicle restarts it. When it expires with no new vehicle, the phase has *gapped out*.

Max Green the ceiling while a conflicting call waits, timed from the moment that call arrives (and reset if it leaves): reaching it is a *max-out* and the green ends even with traffic still flowing. With no conflicting call the timer never runs — the green simply rests.

Yellow / Red Clear the fixed change and clearance intervals that follow every green.

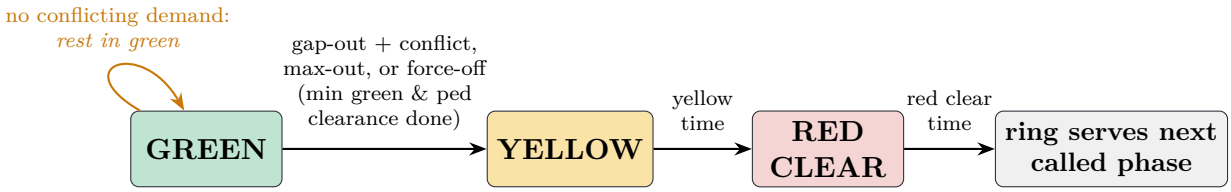


Figure 3: How a phase's green ends. With nothing else calling, the controller simply rests in green on the last served (or SOFT-recalled) phases.

TIP

In-game, sensors detect players and villagers as vehicle proxies. "A vehicle call" means an entity inside the sensor zone mapped to the phase's circuit + movement. Calls are presence-based — gone as soon as the entity leaves the zone — unless the phase's **LOCK** option (Section 5.3) latches them until served.

2.4 Recalls and resting

A **recall** places an automatic call on a phase every cycle, set per phase in the MAP screen's **RCL** column:

RCL	Meaning
`NONE`	Served only on real detection.
`MIN`	Vehicle call every cycle; serves at least Min Green.
`MAX`	Call every cycle and hold the green to Max Green.
`PED`	Pedestrian call (WALK + clearance) every cycle.
`SOFT`	No forced service, but the controller *rests* here when idle.

2.5 Pedestrian timing

A pedestrian service rides on its vehicle phase and has three parts — **WALK**, then the **flashing don't-walk** clearance (**PCI**), then steady don't-walk. Two safety rules are absolute: a started clearance always finishes (nothing may truncate it), and the vehicle green is held at least until the clearance completes.

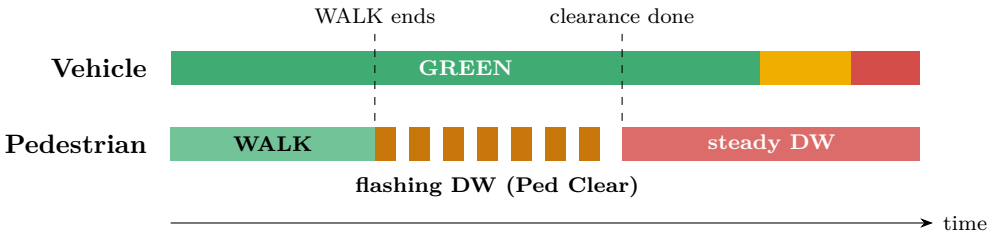


Figure 4: Pedestrian intervals under a vehicle green. After the clearance the phase may keep resting in green (steady don't-walk), or yield.

Two per-phase pedestrian options (the **PED** column, Section 5.3) refine this: **Ped Recall** reserves the WALK every cycle, and **Rest-in-Walk** holds WALK for as long as the controller rests on the phase, running a full clearance before it ever yields. **Delayed Green** (**DGn**, a leading pedestrian interval) holds the vehicle green back while the WALK starts first, giving pedestrians a head start; it applies only when the phase begins with a pedestrian service.

3 Getting Started

3.1 What you need

1. A **traffic signal controller** block.
2. **Signal heads** on each approach, and (recommended) one **sensor** per approach covering its stop-line area.
3. The **signal linker tool**. Right-click the controller to select it, then right-click each signal/sensor to link it to the selected **circuit**. Use *one circuit per approach* — the ASC-3 maps phases onto circuits, so clean circuits make everything easier. Sneak-click unlinks.

3.2 Enter ADVANCED mode and open the panel

Set the controller's operating mode to `ADVANCED`, then open the **ASC-3 panel** from the controller's visual editor (the ASC-3 button), or by clicking the controller block as an op/creative player. The **RUN** LED lights green when the controller is in ADVANCED mode with no fault.

3.3 The 60-second program

1. On MAP, press Load Std 8-Phase. This overwrites the plan with the standard NEMA layout of Figure 1 and auto-assigns phases to your circuits by approach facing.
2. On MAP, disable (**EN** off) any phases your intersection doesn't have — e.g. all four lefts for a simple crossing.
3. On TIMING, set realistic intervals (defaults are sane; Table 2).
4. Give the main street **RCL** = `MIN` (or `SOFT`) so it gets green with no demand.
5. Close the panel. Walk into a side-street sensor zone and watch it get served.

TIP

Only two one-way streets? Enable just $\varphi 2$ and $\varphi 4$. The ring engine happily runs any subset of phases — rings simply skip disabled or uncalled ones.

3.4 What "Load Std 8-Phase" actually programs

The template classifies each circuit by the facing of its first linked through signal: north/south approaches become the **major street**, east/west the **minor street**. Within each street, circuits are taken in circuit-number order and assigned as follows:

Approach circuit	Through	Left*	Left's FYA ref	Max green (thru/left)	Flash
1st N–S circuit	$\varphi 2$	$\varphi 5$	P6	60 s / 15 s	YEL/RED
2nd N–S circuit	$\varphi 6$	$\varphi 1$	P2	60 s / 15 s	YEL/RED
1st E–W circuit	$\varphi 4$	$\varphi 7$	P8	30 s / 15 s	RED/RED
2nd E–W circuit	$\varphi 8$	$\varphi 3$	P4	30 s / 15 s	RED/RED

Table 1: The standard template's circuit-to-phase mapping. *The left phase (movement PROT) is enabled only when its circuit actually has protected/left signal heads, and its FYA reference is set only when the opposing through was also assigned. The **Flash** column is the **FLSH** program written for the through and the left (Section 5.3).

Fine print worth knowing:

- Circuits with no through signals are skipped, and circuits beyond the first two on each street are left unassigned — map extras by hand.
- Re-running the template is safe: it clears every phase assignment first, then reassigns, so it always reflects the current circuit layout.
- It sets only circuit, movement, enablement, the max greens above, and the FYA references. Everything else — min greens, yellows, clearances, walks, recalls, coordination, preempts, overlaps — keeps its current value, so you can re-run it without losing timing work.

4 The Programmer Panel

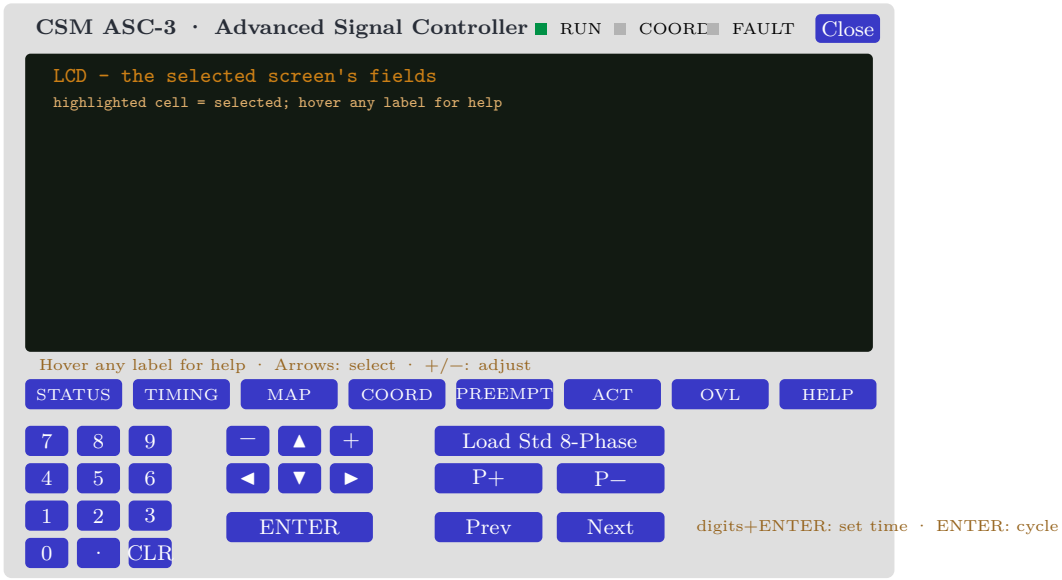


Figure 5: Panel layout: status LEDs, the amber LCD (one screen at a time), the screen-select row, numeric keypad, navigation cluster, and list-management keys.

4.1 Status LEDs

RUN	Green when the controller is in ADVANCED mode and running fault-free.
COORD	Green when a coordinated (fixed-cycle) plan is active (Section 6).
FAULT	Red when a fault is latched (e.g. a misconfigured plan). The fault message appears on STATUS; clear it from the main controller GUI (Clear Faults).

4.2 Editing model

Every screen is a set of editable *cells* on the LCD. One cell is selected (highlighted) at a time.

Input	Effect
▲ ▼ ◀ ▶	Move the selection between cells.
+ / −	Nudge a time value, or cycle a list/toggle.
digits + ENTER	Type a value in *seconds* (decimals allowed) and commit it.
ENTER alone	On a list/toggle cell: advance to the next option.
CLR	Discard the number being typed.
P+ / P−	Add/remove a preempt (on PREEMPT) or overlap (on OVL).
Prev / Next	Select which preempt/overlap is being edited.

Edits apply immediately (they are sent to the server as you make them), and closing the panel also commits a value you were still typing. **Hover any label** — column headers, row labels, LEDs, buttons — for a built-in tooltip.

5 Screen Reference

5.1 STATUS

The overview screen: the controller's mode (and fault message, if any), the **ring/barrier diagram** — each box is *phase:movement*, bright for the main-street group and dim for the side-street group — the coordination summary (FREE, or the running cycle length and offset), and the preempt count. STATUS shows the *program*; it is not a live countdown display.

5.2 TIMING — per-phase intervals

One row per phase (φ1–φ8; dim = disabled), one column per interval, all in seconds. These are the fundamentals from Section 2.3:

Col	Name	Meaning
MnG	Minimum Green	Shortest green once the phase starts, regardless of demand.
Pas	Passage / Gap	Green extension per vehicle call; larger = holds green through bigger gaps.
MxG	Maximum Green	Longest green while a conflicting call waits (max-out).
Yel	Yellow Change	Yellow clearance before red.
Red	Red Clearance	All-red time before the next phase's green.
Wlk	Walk	Steady WALK interval.
PCl	Ped Clearance	Flashing don't-walk countdown; size for the crossing length.
DGn	Delayed Green	Leading pedestrian interval: vehicle green is held this long after WALK starts. 0 = off; ignored when there is no ped service.

Table 2: TIMING columns.

5.3 MAP — what each phase drives

The wiring screen: each phase row maps to a circuit and movement, plus its recall and behavioral options.

Col	Name	Meaning
EN	Enabled	Phase exists in the program (`On`/`off`).
CKT	Circuit	Which linked circuit's heads and sensors this phase uses.
MOVE	Movement	`THRU` / `LEFT` / `PROT` (protected left) / `RGHT` / `PED` — selects both the heads it drives and the sensor zone that calls it.
RCL	Recall	`NONE`/`MIN`/`MAX`/`PED`/`SOFT` (Section 2.4).
PED	Ped opts	`no` / `Ped` (ped recall) / `Walk` (rest-in-walk) / `P+W` (both).
FYA	FYA ref	For a left phase: the *opposing through* whose green flashes this left's yellow arrow (`P`*n*); `--` = protected-only. Section 7.
OPT	Options	`-` / `DE` (dual entry) / `CS` (conditional service) / `D+C` (both).
LOCK	Call memory	`Lock` = latch vehicle calls until served; `-` = presence (call lasts only while the zone sees a vehicle).
FLSH	Flash program	What this phase's movement does in flash: `--` automatic / `YEL` / `RED` / `off`. `n/a` on a `PED` phase. Section 5.3.

Table 3: MAP columns.

Dual entry (DE). Two called phases on the same barrier — one per ring — always run together automatically. DE covers the *uncalled* case: when one ring is serving and the other ring has no call on that barrier, the idle ring brings up its first active DE phase there so one direction isn't left dark. The companion *rides* with the other ring's barrier service: it holds green through the other ring's within-barrier transitions (a through clearing to serve its left, say), and demand waiting on *another* barrier never strips it off early — under coordination the main-street phases are always calling, and the side-street companion rides through that and clears *with* its companion when the barrier's work is done. Only same-barrier demand it conflicts with (a call in its own ring, or its FYA partner) ends it sooner (Section 6.4 covers the timing). A real call always beats dual entry, and DE picks the *first* active DE phase in ring order — flag only the phase you want as the companion.

Conditional service (CS). Once per barrier, an already-passed CS phase that reacquires a call may be re-served before the rings cross — the classic re-served lagging left. After it terminates, the rings park and cross normally.

Locking detector memory (LOCK). By default calls are *presence*-based: a phase calls only while a vehicle is actually inside its detector zone, so a car that clears on a permissive FYA flash — or wanders out of the zone — drops its call, exactly like a stop-bar presence loop. Setting **LOCK** switches the phase to *locking* call memory (the classic pulse-detector behavior): a call registered while the phase is not green is latched and held until the phase is next served, even after the vehicle leaves. Locking guarantees service once anything ever touched the zone, at the cost of occasionally serving a green to a lane the car has already left. Real practice maps directly: leave presence (non-lock) on stop-bar zones — especially FYA lefts — and use LOCK where a missed call is worse than a wasted green. Under coordination a latched call still waits for the phase's permissive window; the latch persists, it doesn't jump the cycle.

Flash program (FLSH). Left alone (`--` on every phase), flash mode uses the controller's automatic rule, which decides yellow-versus-red from the *order the circuits were linked in*: the first circuit flashes yellow and the rest flash red on alternating half-cycles. That lines up at a simple intersection wired in street order, but rarely at one programmed here — the main street may well be C3, and a single circuit often carries both a through and a left.

Setting **FLSH** on any phase switches the controller to a *programmed* flash built from this screen:

- Each programmed phase drives its own movement's heads — so a through can flash `YEL` while

the left on the same circuit flashes `RED`. A through carries its concurrent right turn; a left carries both its protected and permissive lenses.

- Approaches alternate *by barrier*: barrier A (φ1, φ2, φ5, φ6 — the main street and its lefts) lights on one half-cycle, barrier B (φ3, φ4, φ7, φ8 — the side street) on the other. That's the familiar main-versus-cross wig-wag, taken from the phase map rather than from the order you linked the circuits in.
- Anything you don't program flashes **red**, on barrier A's beat. Pedestrian heads are dark and beacons flash as they always do.

The program applies however the controller reaches flash — the `FLASH` mode, nightly fallback, or power-loss fallback. It does not affect the all-red flash or the fault flash, which stay all-red by design.

Load Std 8-Phase fills it in for you: main-street throughs `YEL`, side-street throughs `RED`, every left `RED` — the standard program at a typical four-way.

5.4 ACT — actuation / volume-density

Advanced extension timing, per phase, for busy intersections:

Col	Name	Meaning
Mx2	Max Green 2	Secondary max used *in coordinated operation* when set; 0 = always use **MxG**.
BkG	Bike Min Green	Guaranteed minimum when a bicycle is detected at green start.
AdI	Added Initial	Extra guaranteed initial green *per vehicle* queued at green start (volume-density).
MxI	Max Initial	Cap on the added-initial green. 0 = uncapped.
Gap	Minimum Gap	Floor the passage shrinks toward (gap reduction).
TB4	Time Before Reduce	Green time before gap reduction begins.
TTR	Time To Reduce	Ramp time from full passage down to **Gap**; 0 disables reduction.

Table 4: ACT columns.

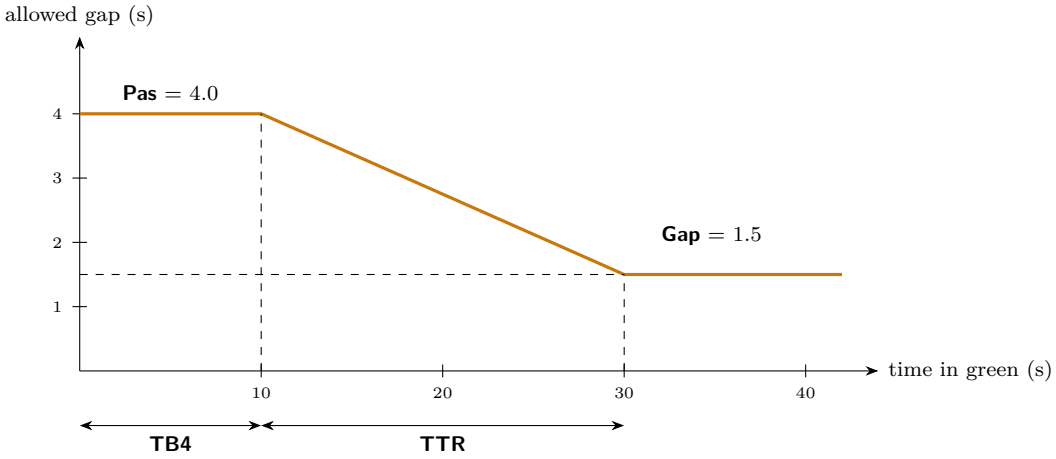


Figure 6: Gap reduction: the longer a green runs, the smaller the vehicle gap that keeps it alive — heavy approaches gap out sooner instead of running to max-out. The ramp starts at the phase's Passage time (**Pas**, set on TIMING, Table 2) and shrinks to this screen's **Gap** floor.

5.5 COORD

Sets the mode (`Free`/`Coordinated`), cycle length, offset, per-phase splits, and the coordinated-phase toggles. An asterisk marks enabled phases (dim rows are disabled). All the theory and programming guidance lives in Section 6.

5.6 PREEMPT — preemption sequences

Preemption suspends normal (or coordinated) operation for a higher authority. [P+] adds a sequence, [Prev]/[Next] select one, and each has:

Enabled	Turn the sequence on/off.
Type	`RAILROAD` > `EMERGENCY VEHICLE` > `TRANSIT PRIORITY`; when several fire at once, the higher type wins.
Trig CKT / MOV	The circuit sensor zone (and movement) that arms the preempt.
Min Dwell	Minimum time in the dwell (hold) state before the preempt may end.
TRACK	Phases served *first* to clear the conflicting path (e.g. traffic queued over a grade crossing).
DWELL	Phases held green for the duration (the train's parallel street, the fire route).
EXIT	Phases served on the way out, before normal service resumes.
SIGNS	*Circuits* (not phases) whose blankout boxes light for the whole sequence — see *Blankout signs* below.

Table 5: PREEMPT fields. Toggle a phase in/out of a set by selecting the row and pressing [ENTER]/[+] on the phase number ([n] = included). **SIGNS** works the same way but lists circuits.

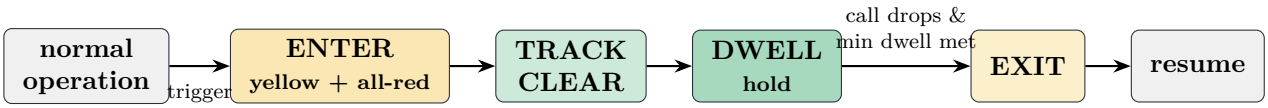


Figure 7: A preemption sequence. Railroad preempts typically use TRACK to flush the crossing, then DWELL on the parallel street; emergency preempts usually skip straight to DWELL on the approach route.

Blankout signs (SIGNS). A blankout box set to the extttTrain or extttNo U-Turn legend has no phase logic of its own — it is dark until a preempt lights it. Select the circuits its boxes are linked to in **SIGNS**, and every such box on those circuits comes up for the entire sequence, from the first yellow of ENTER through the end of EXIT: the extttTrain legend flashes, extttNo U-Turn burns steady. Both go dark again as normal service resumes.

That is the whole rail-crossing package in one place: **TRACK** flushes the crossing, **DWELL** holds the parallel street, and **SIGNS** lights the TRAIN signs on the approaches for as long as the gates are down.

The other two legends are unaffected: extttNo Left Turn / extttNo Right Turn boxes always follow the turns the current phase permits, and extttDon't Walk / extttDo Not Enter boxes link as ordinary pedestrian heads. A box that isn't linked to the controller at all stays fully manual — set its state with the signal head config tool or the box's own configuration screen.

5.7 OVL — vehicle overlaps

An **overlap** is an extra signal output that is green whenever any of its *included* phases is green — the classic use is a right-turn arrow that runs with both its own through phase *and* the compatible cross-street left (e.g. output `RGHT` on φ6's circuit, including φ6+φ7). Rights don't consume phases:

a through phase drives its circuit's right heads by default, and an overlap that outputs to them takes them over. Set the overlap's **Call** phase so a vehicle waiting in the pocket actually summons service (Table 6). Managed like preempts (P+/P−, Prev/Next):

Enabled	Turn the overlap on/off.
Type	`Normal`, or `-Grn/Yel`: additionally forced red while any **MOD** phase is green or yellow.
Out CKT / MOV	Which circuit's heads the overlap drives (`RGHT` = the common right-turn output).
Trail	Lag green: seconds the overlap stays green *after* its included phases leave green, to flush turning vehicles.
Lead	Advance green: seconds it greens *before* an included phase, during the preceding red clearance (within-barrier).
Call	Detector-to-phase assignment: the phase called while vehicles wait in the overlap's output zone. Overlaps are otherwise pure outputs — without this, a movement served only by an overlap generates no demand of its own. `--` = no call.
INCL	The included (protected) phases: green with any of them, yellow during their clearance, red otherwise.
PERM	FYA permissive phases: while any of these is green (and no included phase is), the overlap shows a *flashing yellow arrow*; their clearance shows solid yellow. Empty = classic green/yellow/red overlap.
MOD	Modifier phases (`-Grn/Yel` type only): force the overlap red while green/yellow — e.g. suppress a right-turn arrow during the conflicting pedestrian phase.

Table 6: OVL fields.

FYA overlaps (right-turn flashing yellow arrow). Adding **PERM** phases turns an overlap into a protected/permissive display, the right-turn counterpart of the left-turn FYA: the arrow *flashes yellow* while a permissive phase is green (turn on yield — e.g. the adjacent through, whose concurrent WALK conflicts with the turn) and shows a *protected green arrow* while an included phase is green (e.g. the compatible cross-street left, when the crosswalk is in don't-walk). For the full four-state display, link a 3-section flashing-right lens (as the *flashing-right* signal) with the 1-section right-arrow add-on below it, like an FYA left head; with no lens linked, the output heads themselves flash. Example — right-turn FYA on $\varphi 6$'s approach: **Out CKT** = $\varphi 6$'s circuit, **Out MOV** = `RGHT`, **INCL** = {7}, **PERM** = {6}, **Call** = `P6`. The `-Grn/Yel` modifier still forces red over the flash.

5.8 HELP

A scrollable quick-reference of the same material as this manual: setup, editing, FYA programming, dual-ring concurrency, and coordination. Scroll with the arrows or +/−.

6 Coordination: Cycles, Splits, and Offsets

6.1 Free vs. coordinated

In **FREE** mode every intersection is an island: fully actuated, no schedule. That is ideal in isolation and hopeless on a corridor — platoons released by one signal arrive at the next at random points in its cycle.

COORDINATED mode makes the controller march to a clock, while staying actuated inside the schedule:

Cycle	Length of one full rotation through the phases. Every coordinated signal on a corridor uses the *same* cycle length.
Offset	Where this intersection's cycle starts relative to the shared master clock (in CSM: world time, common to all controllers). Offset *differences* between neighbors build green waves.
Split	Each phase's slice of the cycle — its budget of green + yellow + red clearance.
Coordinated phase(s)	The phase(s) that rest in green and are served every cycle with no call — normally the main-street throughs $\varphi 2/\varphi 6$.

6.2 The cycle as a clock

Think of the cycle as a dial the master clock sweeps once per cycle length. Each phase owns an arc — its **split window**:

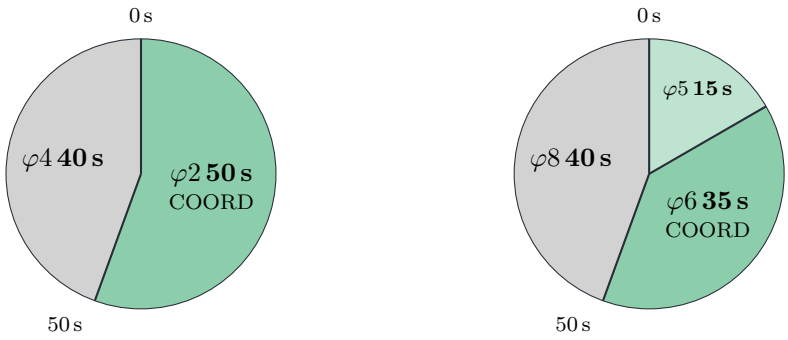


Figure 8: A 90-second cycle as two ring dials (phases 2/4 in ring 1; 5/6/8 in ring 2). The sweep position is the *local cycle* time, (master clock − offset) mod cycle. Both rings cross the barrier together at the 50 s line.

Three rules govern the sweep:

- A **non-coordinated** phase can be *called* only while the sweep is inside its window, and is **forced off** at its **yield point** — set back from the end of its window by that phase's own yellow and red clearance, so the *next* phase goes green exactly as its own window begins. Min green always completes first, and a started pedestrian clearance always finishes.
- A **coordinated** phase is served every cycle with no call, has no max-out, and *rests in green* whenever nothing else may run. It has a yield point of its own (§6.5) and will not give up green before reaching it, even to waiting traffic.
- A split is the whole slice the movement owns — *green and its clearance*. Within that, the green is a *maximum*, not a guarantee: a phase still gaps out early when traffic clears, and the unused remainder reverts to the coordinated phase.

WATCH OUT

Budget each split to hold its whole phase: minimum green (or walk plus pedestrian clearance, whichever is longer), then yellow, then red clearance. A split too short to contain its own phase makes the next phase start late on *every* cycle — time no amount of offset correction can give back — and the controller says so once in the game log.

6.3 Splits are budgeted per barrier

The two rings cross a barrier together, so the cycle is divided barrier by barrier, starting at local cycle 0. On each barrier, each ring's phases share that barrier's time in ring-sequence order:

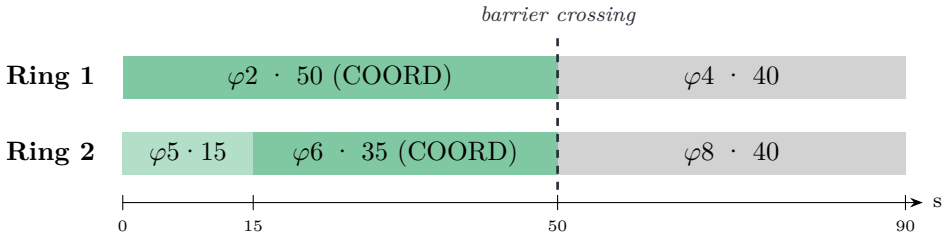


Figure 9: The same plan as Figure 8 as ring timelines. $\varphi 5$ is forced off at 15 s and $\varphi 6$ comes up; both rings cross to the side street together at 50 s.

Two programming rules follow from this:

1. **Match each barrier's total across the rings.** In Figure 9, $\varphi 5 + \varphi 6 = \varphi 2 = 50$ on barrier A and $\varphi 8 = \varphi 4 = 40$ on barrier B. If the totals differ, the engine gives the barrier the *longer* total and stretches the shorter ring's phases to fill it, so they run longer than you typed. The game log warns you once.
2. **Make the barriers total the cycle.** $50 + 40 = 90$. If they don't, every barrier is scaled by cycle ÷ barrier total so the windows still tile the cycle (a split of 0 means "an even share of its ring"). That is a safety net, not a feature: every split then runs a different length from what you typed.

TIP

Sanity check: because the two rings time *in parallel*, a ring with a phase on every barrier has its own full cycle budget. On a 90 s cycle, ring 1's splits total 90 **and** ring 2's splits total 90 — so all eight splits together total **180 s, twice the cycle**. That is parallelism, not overbooking: while $\varphi 2$ owns 0–50 in ring 1, $\varphi 5$ and $\varphi 6$ split that same 0–50 in ring 2. The R1/R2 tag beside each phase on the COORD screen shows its ring, and A/B its barrier. If you instead spread the 90 s across all eight phases, the barriers total only 45, and scaling makes every green about *twice* what you typed.

T intersections: a ring with nothing on a barrier. A three-leg intersection often has no ring-2 phase on the side-street barrier. Take $\varphi 1{+}\varphi 2{+}\varphi 4$ in ring 1 and only $\varphi 6$ in ring 2. Ring 2 cannot total the cycle, and it does not need to. It waits at the barrier while ring 1 serves the side street, so $\varphi 6$ takes only barrier A's time:

	Barrier A		Barrier B	Cycle
Ring 1	$\varphi 1 = 10$	$\varphi 2 = 45$	$\varphi 4 = 35$	90
Ring 2	$\varphi 6 = 55$	$(= \varphi 1 + \varphi 2)$	(waits)	55

Do *not* set $\varphi 6$ to 90 to make ring 2 "total the cycle". That makes barrier A 90 s long against 35 s for barrier B, and the scaled cycle leaves the side street about 25 s. The flipped layout ($\varphi 2{+}\varphi 4$ / $\varphi 5{+}\varphi 6$) works the same way: $\varphi 2 = \varphi 5 + \varphi 6$.

WATCH OUT

A split must cover the whole slice — green **plus** yellow **plus** red clearance — and any WALK + pedestrian clearance served in it. A 10 s split with a 3.5 s yellow and 1.5 s red clear leaves only ~5 s of green.

6.4 Dual entry under coordination

Splits **never add** across a dual-entry pair. The companion comes up at the same moment as the called phase, holds green with it — the coordinated phases' standing calls on the other barrier

never strip it off early — and clears *with* it when the barrier's work is done; while riding along, the companion's own split and force-off are ignored — they apply only when that phase is served on its own call.

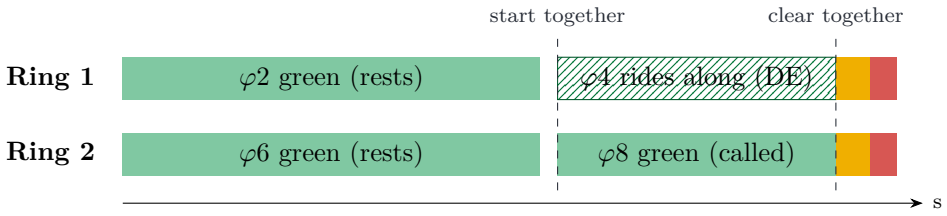


Figure 10: "φ8 has a 40 s split and its DE companion φ4 has 40 s too" — the result is one shared window (here ended early by φ8 gapping out), never 40 + 40.

6.5 Offsets and green waves

An offset by itself means nothing; the *differences* between neighboring offsets are the tool. To build a green wave, give each intersection its upstream neighbor's offset **plus the travel time between them** — a platoon released by the first signal then arrives everywhere on green. A **time–space diagram** shows it directly:

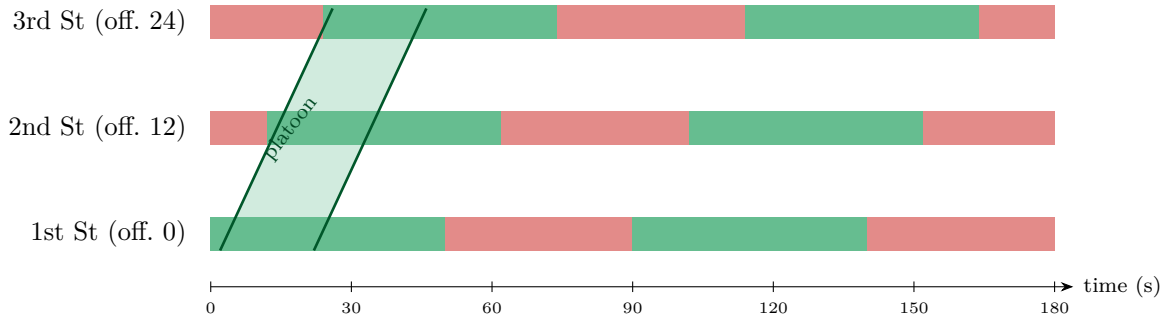


Figure 11: A green wave on a 90 s cycle: 12 s of travel time between signals, offsets 0 / 12 / 24. The platoon (diagonal band) rides green the whole way.

Guidelines:

- **Same cycle length everywhere.** Different cycle lengths drift through every alignment; coordination between them is impossible. Size the common cycle for the busiest intersection.
- One-way corridors are easy: offset = cumulative travel time. Two-way corridors are a compromise — start from the heavier direction's travel times.
- Offsets wrap: on a 90 s cycle, 0 and 90 are identical.

Getting back on offset

Things knock a controller off its offset: a preemption parks normal operation, a chunk reload rebuilds the ring engine, an operator drops the unit to flash and back. The controller then has to walk its cycle back to where the master clock says it should be.

It does so by **dwelling on the coordinated phase**. The coordinated phase is the slack in the plan, so all correction is taken there:

- It will not surrender green *early*. Until its yield point, gap-out and waiting cross traffic cannot end it — it holds green, past max green, rather than giving time away and shifting the rest of the cycle.
- A coordinated phase that finds itself *outside* its own window holds green until the yield point comes round again, absorbing the whole error in one long green.
- The one exception: outside its own window, it gives way when a waiting phase's window is *about to open*. After an early return (the side street gapped out), the main street comes up

early and would otherwise sit on a lead left's window at the top of the cycle, pushing the left to run after the side street instead.

Each non-coordinated phase is served at most once per window, so a side street on recall that gaps out early gives its leftover time back instead of calling again.

Correction is therefore **add-only**: it lengthens the coordinated green and never shortens a side street's split or a pedestrian interval to catch up. A badly displaced controller is back in step within a cycle or two, having only ever over-served the arterial.

TIP

While a controller is recovering, the coordinated green *starts* late and runs across the top of the cycle — so "phase 2 turned green at 0 s" is not the test of a healthy plan. What matters is that the coordinated phase is **green at** the offset, and that each side street **starts** at its own split window. Both are true once the controller is in step.

6.6 Programming recipe

On COORD, at each intersection:

1. **Cycle** — the shared corridor cycle length.
2. **Offset** — 0 at the reference intersection; travel-time based at the rest.
3. **Splits** — a value for every enabled phase (marked `*`). Match each barrier's total across the rings and check the barriers sum to the cycle; a ring with no phase on a barrier is left out of that barrier (§6.3). Size each split to hold the phase *and* its clearance: min green (or walk plus ped clearance, whichever is longer), then yellow, then red clear.
4. **COORD** toggles — mark the main-street phases (usually $\varphi 2{+}\varphi 6$).
5. **Mode** — flip `Free` → `Coordinated`. The COORD LED lights while the plan runs.

Supporting settings: **Mx2** on ACT raises the max green used under coordination without loosening free operation; coordinated phases need no recall (coordination already serves them every cycle); and WALK + ped clearance + yellow + red clear must fit inside each phase's split or that phase will push the next one late on every cycle (safety intervals always finish). The controller writes a one-line advisory to the game log naming any phase whose split cannot hold it.

6.7 Worked example

Four-way through traffic plus one protected left ($\varphi 5$), 90 s cycle, main street coordinated — the plan drawn in Figures 8 and 9:

Phase	Movement	Ring	Split	COORD
$\varphi 2$	Main through (SB)	1	50	✓
$\varphi 4$	Cross through	1	40	
$\varphi 5$	Main left (SB)	2	15	
$\varphi 6$	Main through (NB)	2	35	✓
$\varphi 8$	Cross through	2	40	

Ring 1: $50 + 40 = 90$ ✓ Ring 2: $15 + 35 + 40 = 90$ ✓ Groups: $\varphi 5 + \varphi 6 = \varphi 2$, $\varphi 8 = \varphi 4$ ✓

Each cycle: $\varphi 5$ leads with up to 15 s (skipped entirely with no left-turner — $\varphi 6$ simply comes up early and the main street gets more green), the throughs rest in green until second 50, both rings cross the barrier, and the cross street runs its 40 s window until the wrap forces it off.

7 Flashing Yellow Arrows (FYA)

An FYA left gives drivers a *permissive* turn (yield to oncoming traffic) whenever the opposing through is green, on top of its *protected* arrow phase.

7.1 The hardware

An FYA head is **two linked blocks** stacked as one signal: the 3-section flashing-left lens (linked as the *flashing-left* signal) with the 1-section green-arrow add-on (linked as the *left* signal) mounted below it.

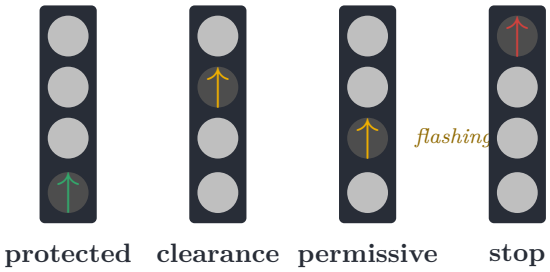


Figure 12: The four FYA states: green arrow during the protected phase, solid yellow during its clearance, flashing yellow while the opposing through is green, red otherwise. (Top three sections = the 3-section lens; bottom = the green-arrow add-on.)

7.2 Programming

On MAP, for the left phase: set **MOVE** to `PROT` (protected left), assign its approach's **CKT**, and set the **FYA** column to the **opposing through phase** — the standard pairings are φ1→`P2`, φ5→`P6`, φ3→`P4`, φ7→`P8`. Set **FYA** to `--` for a protected-only left (no flash).

The controller then arbitrates automatically, including under coordination: the flash runs while the opposing through is green, clears concurrently with it, and is held correctly across cycle wrap and into an imminent protected green.

8 Saving and Loading Whole Plans

A controller's whole program — every phase, the ring sequences, all four coordination patterns and the time-of-day schedule, overlaps, preempts and transit priority plans, plus the controller's red+amber and green flash times — can be taken out and put back as one JSON file, in the `csm-asc3-plan` format that the SPATT timing tool writes and reads (version 2; version 1 files load too). Times are in ticks, as everywhere on the panel.

8.1 From the panel

Copy Plan puts the program on the clipboard. **Paste Plan** loads the plan on the clipboard into the controller, as one edit: the controller resets and starts the new program cleanly, exactly as after any panel edit. This works on a server too, and needs the same permission as any other edit.

8.2 From the command line

Look at the controller (or give its position) and run `/csm signal export <name>` to write `<world>/csm/timing/`
or `/csm signal import <name>` to load it. Both need operator permission.

8.3 What a plan changes

A plan is applied on top of what the controller has. Anything the plan leaves out keeps its value: a plan made in SPATT carries timing but not wiring, so circuits, movements and flash settings stay as you set them. Phases the plan lists are set; phases it does not list are switched off. Pattern slots it does not list are kept, and overlaps, preempts and priority plans are replaced only when the plan includes them. Times are clamped to 600 s.

A plan that is not valid changes nothing, and the panel or command says what is wrong (for example `phases[2].recall is "SOMETIMES"; it must be one of NONE, MINIMUM, MAXIMUM, PEDESTRIAN, SOFT`). A plan that loads but does not yet suit the controller's wiring is reported as a warning, and the controller faults with that message until the wiring is fixed, as it would after the same edit on the panel.

Loading a plan this way is preferred to `/blockdata`: a plan written with `/blockdata` keeps the running engine instead of starting it afresh.

9 Cookbook

9.1 Two one-way streets (the minimal program)

Enable $\varphi 2$ (main) and $\varphi 4$ (cross) only; both `THRU`, each on its approach circuit. $\varphi 2$ **RCL**=`MIN`, $\varphi 4$ on detection. That's a complete actuated intersection. To coordinate: splits 50/40 on a 90 cycle, COORD toggle on $\varphi 2$, Mode → Coordinated.

9.2 Four-way crossing, no lefts

Enable $\varphi 2$, $\varphi 4$, $\varphi 6$, $\varphi 8$ (all `THRU`). Put **DE** on $\varphi 6$ and $\varphi 8$ so a lone car on one approach still lights both directions of its street. Recall the main pair (`MIN` or `SOFT`).

9.3 Main street with protected + permissive lefts

Standard 8-phase: Load Std 8-Phase, then set each left's **FYA** column (Section 7). Give the lefts short splits and consider **CS** (conditional service) on lagging lefts that refill late. Single left-turner traffic clears on the flash alone; the protected arrow only comes up on a real queue.

9.4 The full build: all 8 phases + right-turn FYAs

Everything the controller has on one intersection: the 8-phase program above, plus FYA right-turn signals on both main-street approaches that *flash* during their own through (whose concurrent WALK crosses the turn path) and show a *protected green arrow* alongside the compatible side-street left.

Per main-street approach, link a flashing-right lens + right-arrow add-on and a right-lane sensor zone, then add one overlap each:

Overlap	Out CKT	Out MOV	INCL	PERM	Call
SB right arrow	$\varphi 2$'s circuit	`RGHT`	{3}	{2}	`P2`
NB right arrow	$\varphi 6$'s circuit	`RGHT`	{7}	{6}	`P6`

Table 7: Right-turn FYA overlaps for the full build. For side-street arrows the same pattern continues: **INCL** is the phase right after the approach's through in ring order ($\varphi 4 \to \{5\}$, $\varphi 8 \to \{1\}$) — that cross-street left discharges beside the right turn without crossing it.

Each arrow then cycles: flashing yellow while its through runs, solid yellow with the through's clearance, protected green through the cross-street left, red otherwise — and **Call** means a lone car in the pocket still summons service.

9.5 Bicycles on the arterial

Set **BkG** (ACT) on the through phases bikes ride with — a bicycle standing in the circuit's protected/left detection zone at the moment the green starts guarantees that much green regardless of how quickly cars gap out. Size it to the crossing (8–12 s is typical). Pair it with gap reduction

(**Gap**/**TB4**/**TTR**) so the extra minimum doesn't turn into sloppy long greens, and with **Mx2** if the corridor is coordinated.

9.6 An exclusive pedestrian phase (and ped overlaps)

A phase with **MOVE** = `PED` serves only a crosswalk: give it the circuit whose pedestrian heads and buttons are linked, set **Wlk** and **PCl**, and it is called by that circuit's buttons and served as a normal ring phase — every vehicle movement waits while it runs. Where a crosswalk should instead ride along with several vehicle phases, use a *ped overlap*: an OVL entry with **Out MOV** = `PED` walks its heads while any **INCL** phase serves WALK and runs their clearance countdowns.

9.7 A railroad crossing

On PREEMPT: **Type** = `RAILROAD`, **Trig CKT/MOV** = the sensor zone on the approach track, **TRACK** = the phase(s) whose green flushes queued traffic off the crossing, **DWELL** = the street parallel to the tracks, **EXIT** = the throughs, and **Min Dwell** roughly the time the gates stay down. A train preempts everything — coordination included — and normal service resumes through the exit phases.

Put a blankout box set to the `Train` legend on each approach to the crossing, link it to that approach's circuit, and tick those circuits in **SIGNS** (Section 5.6): the signs flash for exactly as long as the preempt runs, and are dark the rest of the time.

9.8 A coordinated corridor

Pick the busiest intersection; time it in FREE until the splits feel right; read those greens off as splits. Apply the same cycle everywhere (Section 6.6), measure travel time between signals (walk it — ∼1 block/s sprinting is a usable proxy), and set offsets cumulatively. Verify with the COORD LED and by riding the wave yourself.

10 Troubleshooting & FAQ

Symptom	Likely cause / fix
FAULT LED lit	Misconfigured plan (e.g. phase on a missing circuit). Read the message on STATUS, fix, then Clear Faults in the main controller GUI.
A phase never gets served	**EN** off, wrong **CKT**/**MOVE** on MAP, sensor not linked/covering the lane — or (coordinated) its calls fall outside its split window.
Side street served instantly with nobody there	A recall (**RCL**) or ped recall you didn't intend; also check **DE** isn't flagged on a phase you meant to stay call-only.
Left comes up green with no queue	It has **DE** or a recall; for FYA lefts a single car is expected to clear on the flash instead.
A car in a right-turn pocket (overlap output) waits forever	The overlap has no **Call** phase, so its detection places no demand — set **Call** to an included phase (usually the adjacent through).
Green shorter than its split (coordinated)	Normal: it gapped out and returned the rest to the coordinated phase. Also check yellow + red clear fit inside the split.
Phase runs *longer* than the split you typed	Ring splits don't total the cycle, so they were scaled up (Section 6.3).
Rings cross the barrier at different times; one direction dark	Concurrent groups mismatched across rings ($\varphi 5 + \varphi 6 \neq \varphi 2 \ldots$).
Green wave arrives on red	Offset differences ≠ travel time, or cycle lengths differ between intersections.
Cycle occasionally runs long (coordinated)	A WALK + ped clearance doesn't fit its phase's split; safety intervals always finish.
Can't change a value	Select the cell first (arrows), then +/−, or digits + ENTER for times. List cells (mode, movement, recall) cycle with ENTER/+.

Table 8: Common issues.

Is STATUS live? It shows the programmed plan (mode, ring map, coordination, preempts), not a per-second countdown.

Do I need sensors everywhere? Only on approaches you want actuated. A recall (`MIN`) substitutes for detection on approaches that should always be served.

What counts as a vehicle? Players and villagers inside a linked sensor's zone, per the movement mapping (through/left/right totals; `PED` uses pedestrian buttons).

A Glossary

Actuated Operation driven by detection: phases are served on call and end when demand gaps out.

Barrier The line both rings must cross together, separating the main-street phase group from the side-street group.

Call A request for service on a phase — from a sensor, a pedestrian button, or a recall.

Coordinated phase A phase that rests in green and is served every cycle without a call; the anchor of a coordinated plan.

Cycle length Duration of one complete rotation through the phases; shared by every signal on a coordinated corridor.

Dual entry (DE) Serving an uncalled companion phase so both rings show green on the active barrier; the companion rides with the called phase and clears with it (cross-barrier demand never strips it off early).

Conditional service (CS) Re-serving an earlier phase on the same barrier (once per barrier) when it reacquires a call before the rings cross.

Force-off The hard end-of-green a coordinated plan imposes on a non-coordinated phase when its split window closes.

FYA Flashing yellow arrow: a permissive left indication shown while the opposing through is green.

Gap-out A green ending because the passage timer expired with no new vehicle.

Locking memory (LOCK) Per-phase call memory: a call registered while the phase is not green is latched until served, even after the vehicle leaves the zone. Off = presence (the call lasts only while the vehicle is detected).

Max-out A green ending because it reached Max Green with a conflicting call waiting.

Offset The start of an intersection's cycle relative to the shared master clock (world time).

Overlap An extra signal output that runs green with a set of included phases (e.g. a right-turn arrow).

Passage The vehicle-extension (gap) timer added per detection.

Permissive window The portion of the cycle in which a non-coordinated phase may be called and served — its split, placed on the ring timeline.

Phase The controller's unit of right-of-way: a circuit + movement plus its timing.

Preemption A sequence (railroad / emergency / transit) that suspends normal operation: enter → track clear → dwell → exit.

Recall An automatic call placed on a phase every cycle (`MIN`/`MAX`/`PED`), or a resting preference (`SOFT`).

Ring One of two parallel phase sequences; at most one phase per ring is green at a time.

Split A phase's slice of the cycle (green + yellow + red clearance), normalized per ring.

B Cross-reference to the codebase

For modders and the curious — where each piece lives, under the package `com.micatechnologies.minecraft.csm.trafficsignals`:

`AdvancedSignalControllerGui`	The ASC-3 panel (all screens, cells, tooltips).
`TileEntityTrafficSignalController`	The controller block entity; applies panel edits (`applyAdvancedConfig`).
`logic.RingBarrierState`	The per-tick dual-ring state machine: calls, greens, barriers, coordination windows, dual entry, conditional service. Yield points and the add-only dwell live here (`pastYieldPoint`, `acceptanceOpen`, `coordYieldReached`, `findSplitShortfall`).
`logic.TrafficSignalProgrammedPhasePlan`	The stored program: phases, ring sequences, coordination, preempts, overlaps.
`logic.TrafficSignalCoordinationPlan`	Cycle/offset/splits/coordinated phases.
`logic.TrafficSignalPreempt`	One preemption sequence.
`logic.TrafficSignalProgrammedOverlap`	One vehicle overlap.
`logic.AdvancedActuationTiming`	Volume-density math (added initial, gap reduction, Max 2).

See also `assets/docs/TRAFFIC_SIGNAL_SYSTEM.md` for the controller and signal-system architecture.